

# DSA 8070 R Session 6: Reduced-Rank Regression and Repeated Measures Analysis

Whitney Huang, Clemson University

## Contents

|                                                                 |    |
|-----------------------------------------------------------------|----|
| Reduced-Rank Regression . . . . .                               | 1  |
| Multivariate Regression Refresher . . . . .                     | 1  |
| Reduced-Rank Regression (RRR): SVD-Based Estimator . . . . .    | 3  |
| Rank Selection via Cross-Validation . . . . .                   | 4  |
| Compare OLS vs. RRR (Predictive Performance) . . . . .          | 6  |
| Inspect Low-Rank Structure . . . . .                            | 7  |
| Repeated Measures Analysis . . . . .                            | 9  |
| Dog Experiment . . . . .                                        | 9  |
| Split-plot ANOVA . . . . .                                      | 9  |
| MANOVA . . . . .                                                | 11 |
| Mixed Model with AR(1) temporal correlation structure . . . . . | 12 |

## Reduced-Rank Regression

### Multivariate Regression Refresher

We model a multivariate response matrix  $Y \in \mathbb{R}^{n \times q}$  with predictors  $X \in \mathbb{R}^{n \times p}$ :

$$Y = XB + E,$$

where  $B \in \mathbb{R}^{p \times q}$  is the coefficient matrix.

A naive approach fits each response column separately (OLS for each). But this **ignores cross-response correlation**.

```
# Set the seed for reproducibility
set.seed(123)

# Specify the sample size, number of predictors, and number of responses
n <- 200
p <- 6
q <- 4

# Generate the n x p design matrix:
```

```

# one intercept and p - 1 independent N(0,1) predictors
X <- cbind(1, matrix(rnorm(n * (p - 1)), n, p - 1))
colnames(X) <- c("Intercept", paste0("X", 1:(p - 1)))

# Construct a true coefficient matrix B with rank 2
# using the factorization B = AC
r_true <- 2
A <- matrix(rnorm(p * r_true), p, r_true)
C <- matrix(rnorm(r_true * q), r_true, q)
B_true <- A %*% C # p x q coefficient matrix

# Specify correlation among the q response errors
Sigma_eps <- matrix(c(1.0, 0.6, 0.3, 0.2,
                      0.6, 1.0, 0.5, 0.4,
                      0.3, 0.5, 1.0, 0.7,
                      0.2, 0.4, 0.7, 1.0), q, q)

# Generate n independent multivariate Gaussian error vectors
library(MASS)
E <- MASS::mvrnorm(n, mu = rep(0, q), Sigma = 2 * Sigma_eps)

# Generate responses from the multivariate regression model Y = XB + E
Y <- X %*% B_true + E
colnames(Y) <- paste0("Y", 1:q)

# Fit a separate OLS regression to each response as the baseline model
# "-1" avoids adding another intercept since X already contains one
ols_fits <- vector("list", length = q)
for (j in 1:q) {
  ols_fits[[j]] <- lm(Y[, j] ~ X - 1)
}
names(ols_fits) <- colnames(Y)

# Collect the estimated regression coefficients into a p x q matrix
B_ols <- array(unlist(lapply(ols_fits, coef)), dim = c(p, q))

# Compare the estimated coefficient matrix with the true coefficient matrix
B_ols

```

```

##           [,1]      [,2]      [,3]      [,4]
## [1,] 1.29633691 0.3063739 -0.5678854 0.1460612
## [2,] 2.32378304 0.6533733 -3.0533527 4.0340122
## [3,] 0.36436374 0.1528000 -0.8591614 1.1869485
## [4,] 0.07241715 0.1164822 0.6869218 -0.7811949
## [5,] 4.43669300 1.1061847 -4.1532537 4.1316973
## [6,] 0.03901217 0.4363386 -3.4027784 5.2680089

```

B\_true

```

##           [,1]      [,2]      [,3]      [,4]
## [1,] 1.33072102 0.25707090 -0.5417158 0.1524423
## [2,] 2.40068397 0.67164166 -3.2856474 3.9295441
## [3,] 0.34329260 0.13196539 -0.8687432 1.1934462

```

```
## [4,] -0.04855480 -0.05568266 0.5339427 -0.8195902
## [5,] 4.41898000 1.06179603 -4.1101136 4.1652634
## [6,] 0.06024535 0.30995319 -3.3372124 5.2514408
```

## Reduced-Rank Regression (RRR): SVD-Based Estimator

**Idea:** Fit full multivariate OLS, then project fitted values to rank  $r$  via SVD.

Algorithm:

1.  $\hat{B}^{OLS} = (X^\top X)^{-1} X^\top Y$
2.  $\hat{Y} = X \hat{B}^{OLS}$
3. SVD:  $\hat{Y} = U \Sigma V^\top$
4. Keep top  $r$  right singular vectors  $V_r$ , set  $\hat{B}^{RRR} = \hat{B}^{OLS} V_r V_r^\top$

```
# Fit a reduced-rank regression (RRR) model with rank r
RRR_fit <- function(X, Y, r) {
  # Step 1: Fit the unrestricted multivariate OLS model
  # B_OLS = (X'X)^(-1)X'Y
  XtX <- crossprod(X)
  ridge <- 1e-8 * diag(ncol(X)) # Small ridge term for numerical stability
  B_ols <- solve(XtX + ridge, crossprod(X, Y))

  # Step 2: Compute the OLS fitted response matrix
  # Y_hat = X B_OLS
  Yhat <- X %*% B_ols

  # Step 3: Apply SVD to the fitted responses
  # Y_hat = U D V'
  svdY <- svd(Yhat)

  # Step 4: Retain the first r right singular vectors
  # These define the dominant r-dimensional subspace in response space
  Vr <- svdY$v[, 1:r, drop = FALSE]

  # Step 5: Project B_OLS onto the rank-r response subspace
  # B_RRR = B_OLS V_r V_r'
  B_rrr <- B_ols %*% Vr %*% t(Vr)

  # Return the OLS and reduced-rank estimates
  out <- list(B_ols = B_ols, B_rrr = B_rrr,
             Vr = Vr, svd = svdY)
  return(out)
}

# Predict the multivariate response for new observations
predict_multivar <- function(Xnew, B) {
  Xnew %*% B
}
```

Try a few ranks and compare in-sample fit:

```

# Consider all possible ranks from  $r = 0$  to  $r = \min(p, q)$ 
#  $r = 0$  corresponds to no systematic predictor-response relationship
r_grid <- 0:min(ncol(X), ncol(Y))

# Create a matrix to store the RSS for each candidate rank
RSS_by_r <- matrix(NA_real_, nrow = length(r_grid), ncol = 2)
colnames(RSS_by_r) <- c("r", "RSS")

# Fit RRR models across all candidate ranks
for (i in seq_along(r_grid)) {
  r <- r_grid[i]
  # For  $r = 0$ , set all regression coefficients equal to zero
  if (r == 0) {
    B <- matrix(0, nrow = ncol(X), ncol = ncol(Y))
  } else {
    # For  $r > 0$ , obtain the rank- $r$  RRR coefficient estimate
    B <- RRR_fit(X, Y, r)$B_rrr
  }
  # Compute fitted responses and residual sum of squares
  Yhat <- X %*% B
  RSS_by_r[i, ] <- c(r, sum((Y - Yhat)^2))
}

# Compare model fit across different ranks
RSS_by_r

```

```

##      r      RSS
## [1,] 0 26482.308
## [2,] 1  4007.351
## [3,] 2  1495.498
## [4,] 3  1479.386
## [5,] 4  1473.253

```

## Rank Selection via Cross-Validation

We evaluate predictive performance for candidate ranks using K-fold CV on rows (observations).

```

# K-fold cross-validation for selecting the rank in RRR
cv_rrr <- function(X, Y, ranks, K = 5, seed = 1) {
  # Randomly assign observations to K folds
  set.seed(seed)
  n <- nrow(X)
  folds <- sample(rep(1:K, length.out = n))
  # Store the rank, fold number, and test MSE for each fit
  res <- matrix(NA_real_, nrow = length(ranks) * K, ncol = 3)
  colnames(res) <- c("r", "fold", "mse")
  row_idx <- 1
  # Evaluate each candidate rank using K-fold cross-validation
  for (rr in ranks) {
    for (k in 1:K) {
      # Split the data into training and test sets
      idx_te <- which(folds == k)
    }
  }
}

```

```

idx_tr <- setdiff(seq_len(n), idx_te)
Xtr <- X[idx_tr, , drop = FALSE]
Ytr <- Y[idx_tr, , drop = FALSE]
Xte <- X[idx_te, , drop = FALSE]
Yte <- Y[idx_te, , drop = FALSE]
# Fit the rank-r RRR model using the training data
# For r = 0, use B = 0 as the null model
if (rr == 0) {
  B <- matrix(0, nrow = ncol(X), ncol = ncol(Y))
} else {
  B <- RRR_fit(Xtr, Ytr, rr)$B_rrr
}
# Predict the held-out responses and compute test MSE
Ypred <- Xte %*% B
mse <- mean((Yte - Ypred)^2)
# Store the cross-validation result
res[row_idx, ] <- c(rr, k, mse)
row_idx <- row_idx + 1
}
}
return(res)
}

# Candidate ranks: r = 0, ..., min(p, q)
ranks <- 0:min(ncol(X), ncol(Y))

# Perform 5-fold cross-validation
cv_res <- cv_rrr(X, Y, ranks = ranks, K = 5, seed = 123)

# Compute the average test MSE across the K folds for each rank
cv_summary <- matrix(NA_real_, nrow = length(ranks), ncol = 2)
colnames(cv_summary) <- c("r", "cv_mse")

for (i in seq_along(ranks)) {
  r <- ranks[i]
  cv_summary[i, 1] <- r
  cv_summary[i, 2] <- mean(cv_res[cv_res[, 1] == r, 3])
}

# Compare predictive performance across candidate ranks
cv_summary

```

```

##      r    cv_mse
## [1,] 0 33.102885
## [2,] 1  5.069399
## [3,] 2  1.958754
## [4,] 3  1.996958
## [5,] 4  1.994387

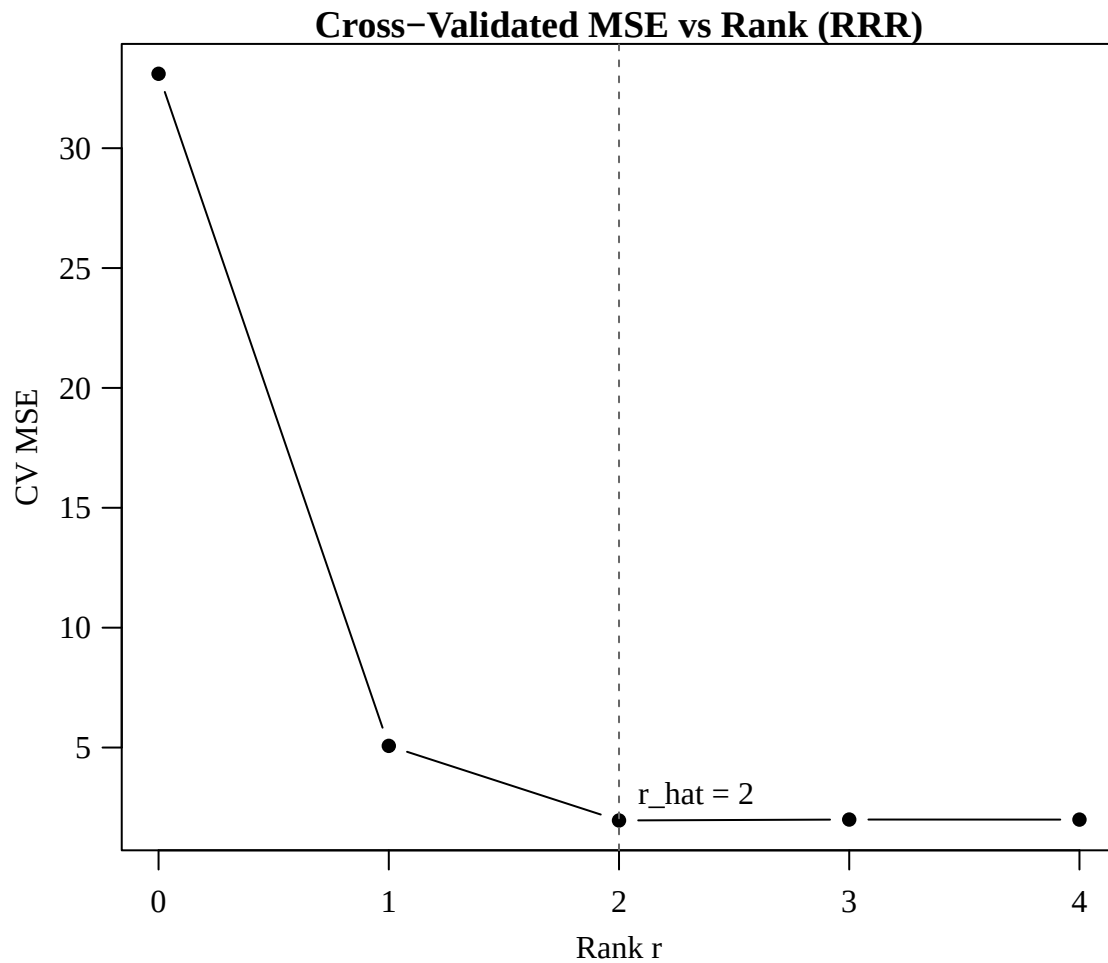
```

Plot CV curve and select  $\hat{r}$ :

```

par(mar = c(3, 3.5, 1, 0.6), mgp = c(2, 0.8, 0), las = 1,
    family = "serif")
plot(cv_summary[,1], cv_summary[,2], type = "b", xlab = "Rank r",
     ylab = "CV MSE", main = "Cross-Validated MSE vs Rank (RRR)",
     pch = 16)
r_hat <- cv_summary[which.min(cv_summary[,2]), 1]
abline(v = r_hat, lty = 2, col = "gray40")
text(r_hat, min(cv_summary[,2]) + 1, labels = paste("r_hat =", r_hat), pos = 4)

```



```
r_hat
```

```
## r
## 2
```

### Compare OLS vs. RRR (Predictive Performance)

Hold out a test set to compare OLS and RRR predictions.

```

# Create an independent train/test split for final model comparison
set.seed(1234)

```

```

# Use 70% of the observations for training and 30% for testing
test_idx <- sample(1:n, size = round(0.3 * n))
train_idx <- setdiff(1:n, test_idx)

Xtr <- X[train_idx, , drop = FALSE]
Ytr <- Y[train_idx, , drop = FALSE]
Xte <- X[test_idx, , drop = FALSE]
Yte <- Y[test_idx, , drop = FALSE]

# Fit the unrestricted multivariate OLS model using the training data
#  $B_{OLS} = (X'X)^{-1}X'Y$ 
B_ols_tr <- solve(crossprod(Xtr) + 1e-8 * diag(ncol(Xtr)),
                 crossprod(Xtr, Ytr))

# Predict the multivariate responses for the test data
# and compute the out-of-sample prediction MSE
Yhat_ols <- Xte %*% B_ols_tr
mse_ols <- mean((Yte - Yhat_ols)^2)

# Fit the reduced-rank regression model using the selected rank
r_use <- max(1, r_hat) # Ensure that the fitted RRR model has rank >= 1
B_rrr_tr <- RRR_fit(Xtr, Ytr, r = r_use)$B_rrr

# Predict the test responses using the reduced-rank coefficient estimate
# and compute the out-of-sample prediction MSE
Yhat_rrr <- Xte %*% B_rrr_tr
mse_rrr <- mean((Yte - Yhat_rrr)^2)

# Compare the test-set predictive performance of OLS and RRR
# A smaller test MSE indicates better out-of-sample prediction
res_compare <- rbind(
  cbind(Method = "OLS (separate)", Test_MSE = mse_ols),
  cbind(Method = paste0("RRR (r=", r_use, ")"), Test_MSE = mse_rrr)
)

res_compare

```

```

##      Method      Test_MSE
## [1,] "OLS (separate)" "1.90507096441179"
## [2,] "RRR (r=2)"      "1.89022281117133"

```

## Inspect Low-Rank Structure

Visualize the right singular vectors (response-space directions) that RRR uses.

```

# Fit the final RRR model using the selected rank
fit_full <- RRR_fit(X, Y, r = r_use)

# Extract the first r right singular vectors
# Each column of V_r defines a dominant direction in the response space,
# and its entries are the loadings of the original responses on that component
Vr <- fit_full$Vr # q x r_use

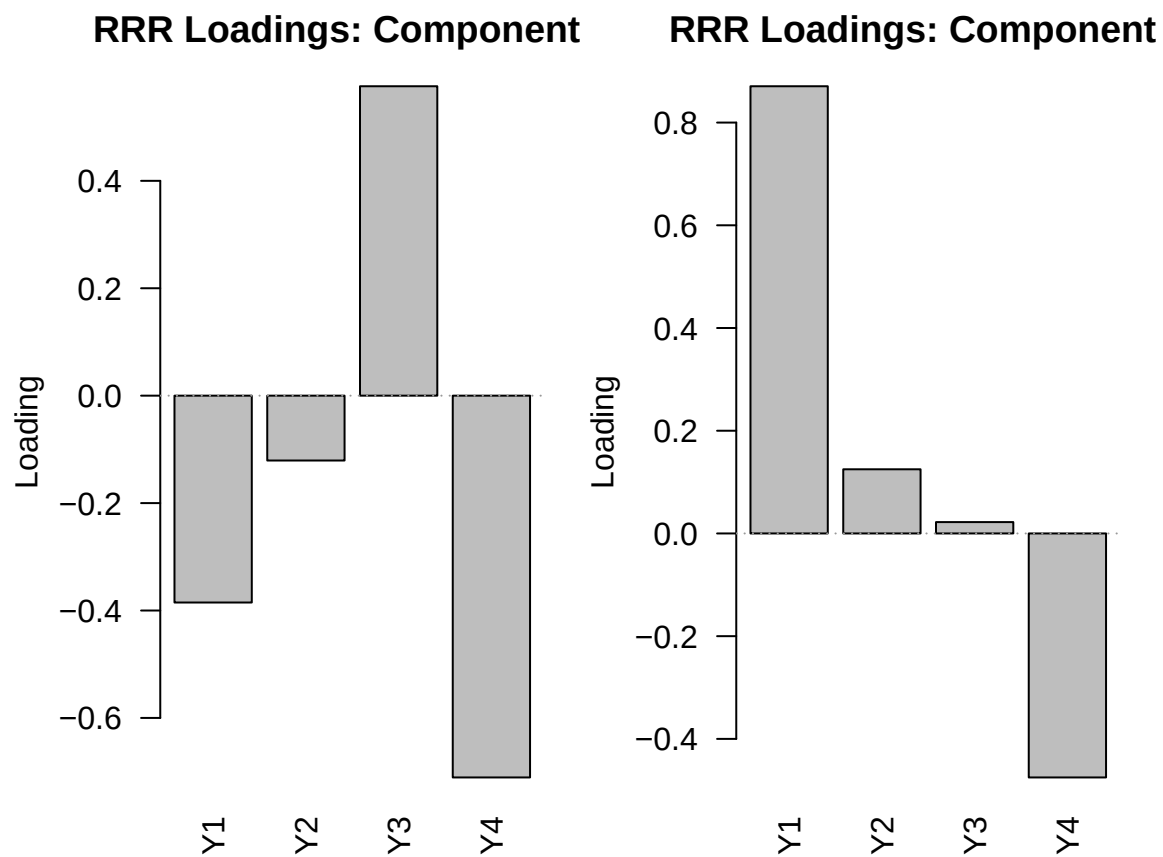
```

```

# Plot the response loadings for each retained RRR component
# Larger absolute loadings indicate responses that contribute more strongly
# to the corresponding response-space direction
op <- par(mfrow = c(1, max(1, ncol(Vr))),
         mar = c(4, 4, 3, 1))

for (j in 1:ncol(Vr)) {
  # Display the loading of each response on component j
  barplot(Vr[, j],
          names.arg = colnames(Y),
          las = 2,
          main = paste("RRR Loadings: Component", j),
          ylab = "Loading")
  # Add a reference line to distinguish positive and negative loadings
  abline(h = 0, col = "gray60", lty = 3)
}

```



```

# Restore the original graphical settings
par(op)

```

## Repeated Measures Analysis

### Dog Experiment

```
# Read the dog experiment data in wide format
# Each row corresponds to one dog, with repeated responses at four time points
dat <- read.table("dog1.txt")

# Create a long-format data matrix:
# 36 dogs measured at 4 time points gives 36 x 4 = 144 observations
temp <- array(dim = c(144, 4))

# Repeat each dog's treatment assignment across the four measurement times
temp[, 1] <- rep(dat$V1, 4)

# Repeat each dog's ID across the four measurement times
temp[, 2] <- rep(dat$V2, 4)

# Create the measurement-time variable:
# responses were recorded at 1, 5, 9, and 13 minutes
temp[, 3] <- rep(c(1, 5, 9, 13), each = 36)

# Stack the four repeated response measurements into a single column
temp[, 4] <- c(dat$V3, dat$V4, dat$V5, dat$V6)

# Convert the long-format matrix to a data frame
dat2 <- data.frame(temp)
names(dat2) <- c("Treatment", "Dog_id", "Time", "Response")

# Treat treatment, dog ID, and measurement time as categorical variables
# Dog_id identifies the experimental subject for repeated measurements
dat2$Treatment <- as.factor(dat2$Treatment)
dat2$Dog_id <- as.factor(dat2$Dog_id)
dat2$Time <- as.factor(dat2$Time)
```

### Split-plot ANOVA

```
# Compute the mean response for each Treatment x Time combination
# Rows correspond to treatments and columns correspond to measurement times
tapply(dat2$Response, list(dat2$Treatment, dat2$Time), mean)
```

```
##           1           5           9          13
## 1 4.111111 4.400000 5.066667 4.722222
## 2 3.600000 4.200000 4.500000 4.225000
## 3 3.644444 4.011111 3.977778 4.044444
## 4 3.540000 3.620000 3.660000 3.460000
```

```
# Plot the treatment-specific mean response profiles over time
# Nonparallel profiles suggest a possible Treatment x Time interaction
par(las = 1, mgp = c(2.2, 1, 0), mar = c(3.6, 3.6, 0.8, 0.6),
```

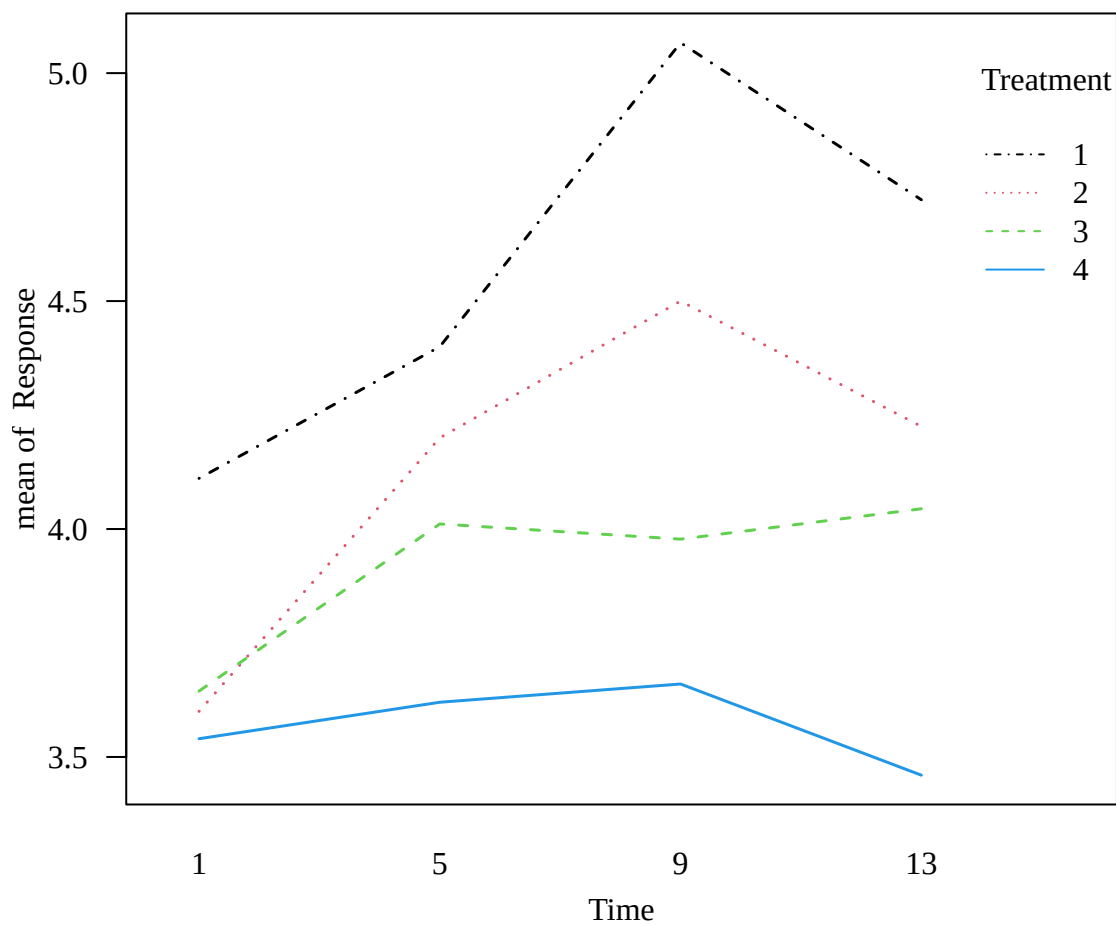
```

family = "serif")

with(dat2, interaction.plot(x.factor = Time,
                           trace.factor = Treatment, response = Response,
                           col = 1:4, lwd = 1.5))

# Fit the split-plot model using a linear mixed-effects model
# Fixed effects:
#   Treatment      = between-subject treatment effect
#   Time           = within-subject time effect
#   Treatment:Time = whether treatment effects change over time
# Random effect:
#   (1 | Dog_id) accounts for repeated measurements on the same dog
library(lmerTest)

```



```

fit <- lmer(Response ~ Treatment * Time + (1 | Dog_id), data = dat2)

# Display the fitted mixed-model results
fit

## Linear mixed model fit by REML ['lmerModLmerTest']
## Formula: Response ~ Treatment * Time + (1 | Dog_id)
## Data: dat2

```

```
## REML criterion at convergence: 239.8063
## Random effects:
## Groups Name Std.Dev.
## Dog_id (Intercept) 0.4798
## Residual 0.4306
## Number of obs: 144, groups: Dog_id, 36
## Fixed Effects:
## (Intercept) Treatment2 Treatment3 Treatment4
## 4.11111 -0.51111 -0.46667 -0.57111
## Time5 Time9 Time13 Treatment2:Time5
## 0.28889 0.95556 0.61111 0.31111
## Treatment3:Time5 Treatment4:Time5 Treatment2:Time9 Treatment3:Time9
## 0.07778 -0.20889 -0.05556 -0.62222
## Treatment4:Time9 Treatment2:Time13 Treatment3:Time13 Treatment4:Time13
## -0.83556 0.01389 -0.21111 -0.69111
```

```
# Test the fixed effects of Treatment, Time, and Treatment x Time
anova(fit)
```

```
## Type III Analysis of Variance Table with Satterthwaite's method
## Sum Sq Mean Sq NumDF DenDF F value Pr(>F)
## Treatment 3.3396 1.11319 3 32 6.0038 0.002297 **
## Time 6.2043 2.06811 3 96 11.1540 2.404e-06 ***
## Treatment:Time 3.4397 0.38219 9 96 2.0613 0.040573 *
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

## MANOVA

```
# Treat the four repeated measurements as a multivariate response vector
# and fit a one-way MANOVA to compare the treatment groups
out <- manova(cbind(V3, V4, V5, V6) ~ as.factor(V1), data = dat)

# Test whether the mean response profiles differ among treatments
# using Wilks' Lambda as the multivariate test statistic
summary(out, test = "Wilks")
```

```
## Df Wilks approx F num Df den Df Pr(>F)
## as.factor(V1) 3 0.48452 2.022 12 77.018 0.03316 *
## Residuals 32
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
# Display the default MANOVA summary
# The default multivariate test in R is Pillai's trace
summary(out)
```

```
## Df Pillai approx F num Df den Df Pr(>F)
## as.factor(V1) 3 0.5978 1.9286 12 93 0.04048 *
## Residuals 32
## ---
## Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

## Mixed Model with AR(1) temporal correlation structure

```
library(nlme)

##
## Attaching package: 'nlme'

## The following object is masked from 'package:lme4':
##
##      lmList

# Fit a repeated-measures model with compound-symmetry (CS) correlation
# The mean model includes Treatment, Time, and their interaction
# CS assumes the same correlation between any two measurements from the same dog
fit1 <- gls(Response ~ Treatment * Time,
            correlation = corCompSymm(form = ~ 1 | Dog_id), data = dat2)

# Display the fitted model and estimated correlation parameter
fit1

## Generalized least squares fit by REML
##   Model: Response ~ Treatment * Time
##   Data: dat2
##   Log-restricted-likelihood: -119.9032
##
## Coefficients:
##      (Intercept)      Treatment2      Treatment3      Treatment4
##      4.11111111      -0.51111111      -0.46666667      -0.57111111
##      Time5          Time9          Time13      Treatment2:Time5
##      0.28888889      0.95555556      0.61111111      0.31111111
##      Treatment3:Time5 Treatment4:Time5 Treatment2:Time9 Treatment3:Time9
##      0.07777778      -0.20888889      -0.05555556      -0.62222222
##      Treatment4:Time9 Treatment2:Time13 Treatment3:Time13 Treatment4:Time13
##      -0.83555556      0.01388889      -0.21111111      -0.69111111
##
## Correlation Structure: Compound symmetry
##   Formula: ~1 | Dog_id
##   Parameter estimate(s):
##      Rho
## 0.5538616
## Degrees of freedom: 144 total; 128 residual
## Residual standard error: 0.6446676

# Fit the same mean model using an AR(1) within-dog correlation structure
# AR(1) allows correlation to decrease as measurements become farther apart in time
fit2 <- gls(Response ~ Treatment * Time,
            correlation = corAR1(form = ~ 1 | Dog_id), data = dat2)

# Display the fitted model and estimated AR(1) correlation parameter
fit2
```

```

## Generalized least squares fit by REML
## Model: Response ~ Treatment * Time
## Data: dat2
## Log-restricted-likelihood: -120.7906
##
## Coefficients:
##      (Intercept)      Treatment2      Treatment3      Treatment4
##      4.11111111      -0.51111111      -0.46666667      -0.57111111
##      Time5          Time9          Time13      Treatment2:Time5
##      0.28888889      0.95555556      0.61111111      0.31111111
## Treatment3:Time5 Treatment4:Time5 Treatment2:Time9 Treatment3:Time9
##      0.07777778      -0.20888889      -0.05555556      -0.62222222
## Treatment4:Time9 Treatment2:Time13 Treatment3:Time13 Treatment4:Time13
##      -0.83555556      0.01388889      -0.21111111      -0.69111111
##
## Correlation Structure: AR(1)
## Formula: ~1 | Dog_id
## Parameter estimate(s):
##      Phi
## 0.5928708
## Degrees of freedom: 144 total; 128 residual
## Residual standard error: 0.6376364

# Compare the two correlation structures using likelihood-based criteria
# The models have the same fixed effects but different within-dog correlation structures
anova(fit1, fit2)

##      Model df      AIC      BIC    logLik
## fit1      1 18 275.8063 327.1429 -119.9032
## fit2      2 18 277.5811 328.9177 -120.7906

```